Codebase Onboarding Agent — Project Report

Created as part of an assignment for Codecademy's Building Agentic AI Applications bootcamp, with Satyajit Pattnaik, Fall 2026.

This project helps you quickly understand any public Python repository on GitHub. It combines **RAG (Retrieval-Augmented Generation)**, to index and search a codebase by meaning, with an **AI agent** that explores files, traces how code connects, and answers questions in plain English.

Contest tracks covered

The contest asks for either track. This project implements both, with the RAG pipeline as the agent's knowledge layer.

Requirement	How this project meets it
Complex knowledge base	A whole software repository, documentation *and* source code, chunked by structure (headings for docs, functions and classes for code). Any public Python repo can be loaded.
Vector database	A Pinecone serverless index (cosine similarity) with one namespace per repository. Vectors persist across sessions.
LangGraph workflow	A LangGraph `StateGraph` with three nodes: `agent`, `tools` (7 tools) and `check_citations`, which verifies every `file:line` and sends faulty answers back for a rewrite. It enforces a 15-call tool limit, falls back to other models on errors, and keeps conversations with an `InMemorySaver` checkpointer.
Clean, user-friendly UI	A Gradio web app that shows each tool call as a collapsible step, shows indexing progress, and links citations to GitHub.
Evaluation	An 8-question test on `pallets/itsdangerous` that checks answers cite the right files with valid line ranges, and that the agent admits when something isn't in the repo. Result: **8/8**.
Code on GitHub + report	`github.com/jvs867/codebase-onboarding-agent` (notebook, README, requirements) and this report.

Goals

- **Any public Python repo:** paste a GitHub link to start.
- **Search by meaning (RAG):** find code and docs by what they do, not just exact keywords.
- **Explore step by step (agent):** choose tools, read files and follow logic across the project instead of guessing from one search.
- **Stay grounded:** cite the exact files and lines used, verified and linked to GitHub, and say so when something isn't in the codebase.
- **Explain clearly, and be easy to use:** plain-language answers in a simple web interface.

System architecture

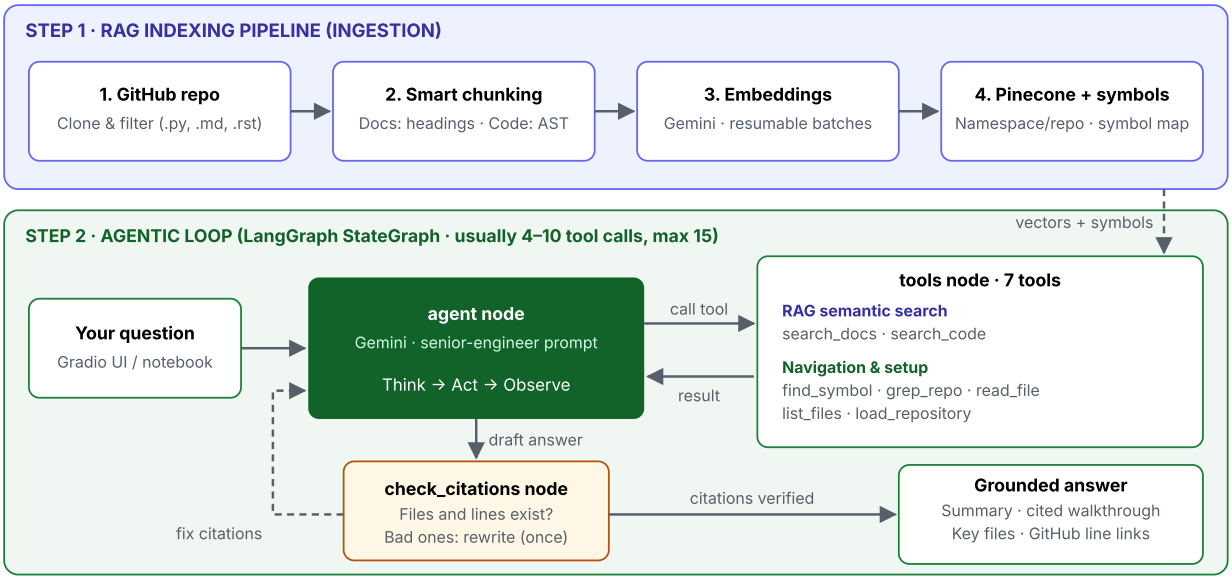


Figure 1: End-to-end architecture. Step 1 (RAG indexing) feeds Step 2, a LangGraph StateGraph that loops between the agent and tools nodes, then checks citations before answering.

Step 1: The RAG pipeline

The first time a repository is loaded, it is prepared for semantic search:

1. **Download:** a shallow clone onto the Colab machine.
2. **Filter:** keep Python files (`.py`) and documentation (`.md` , `.rst`); skip tests, oversized files, build output and third-party folders.
3. **Chunk by structure:** documentation splits at section headings (up to **2,000 characters**, **200-character overlap**). Python code splits with the built-in `ast` module into one chunk per function, method, class overview and module overview (up to **6,000 characters**), so a function is never cut in half.
4. **Embed:** Google's embedding model turns each chunk's meaning into a vector. Requests are sized by estimated tokens, a rate limiter keeps every rolling minute under Gemini's quota, and retries honour Google's requested delay.
5. **Store:** vectors go into Pinecone, one isolated namespace per repository.

> Chunking also builds a lookup table of every function and class and where it is defined, so it runs on every load. Already-indexed repositories **skip steps 4–5**, and an interrupted upload **resumes** with only the missing chunks.

Step 2: The agentic workflow

Standard RAG runs one search before answering. Here an agent sits on top of the RAG pipeline and loops (**think → pick a tool → inspect the result → repeat**) until it has enough evidence. The loop is a LangGraph `StateGraph` :

Node	What it does
`agent`	Gemini reads the conversation and either requests tool calls or writes the answer. Fallback models take over on errors.
`tools`	Runs the requested tools (LangGraph's `ToolNode`) and returns the results to `agent`.
`check_citations`	Checks every `file:line` in the draft answer against the cloned repository (the file exists, the lines are inside it). A faulty answer goes back to `agent` once with the problems listed, e.g. *"`signer.py` does not exist (did you mean `src/itsdangerous/signer.py`?)"*

Conditional edges send `agent` to `tools` when it asks for tools and to `check_citations` when it answers; `check_citations` ends the run or sends the answer back. The graph state counts tool calls, and after 15 the agent must answer without tools. An `InMemorySaver` checkpointer keeps each conversation for follow-ups. The notebook draws the graph (Figure 2), and the citation check appears as a step in the notebook trace and the web UI.

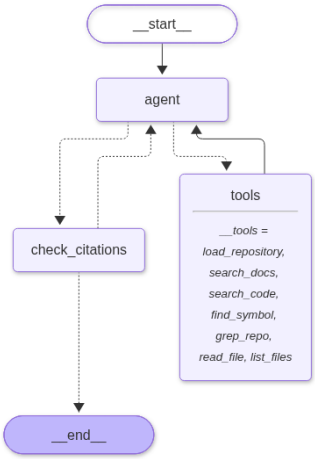


Figure 2: The agent graph as drawn by LangGraph. Solid edges are fixed; dotted edges are conditional.

The agent's 7 tools

Tool	Type	What it does
`load_repository`	Setup	Downloads and indexes a new GitHub project (Step 1).
`search_docs`	**RAG**	Searches documentation by meaning.
`search_code`	**RAG**	Searches code by what it does (e.g. *"code that checks a password"*).
`find_symbol`	Navigation	Jumps to where a function or class is defined.
`grep_repo`	Navigation	Searches for exact text across files, like Ctrl+F.
`read_file`	Navigation	Reads only the lines it needs from a file.
`list_files`	Navigation	Lists the project's folders and files.

How the agent is guided

The system prompt has the agent act like a senior developer onboarding a teammate: never guess, start with the docs, then trace the code from file to file. It usually needs 4–10 tool calls; the graph enforces 15 and a recursion limit is a final stop. Answers have a short summary, numbered steps citing `file:START-END`, and the files to read first. Citations are verified and become links to those lines on GitHub.

Results

All runs use `pallets/itsdangerous` (9 documentation files, 9 Python files, 88 symbols, 105 vectors). At start-up the notebook probes each chat model and uses the first that responds, here `gemini-3.1-flash-lite`.

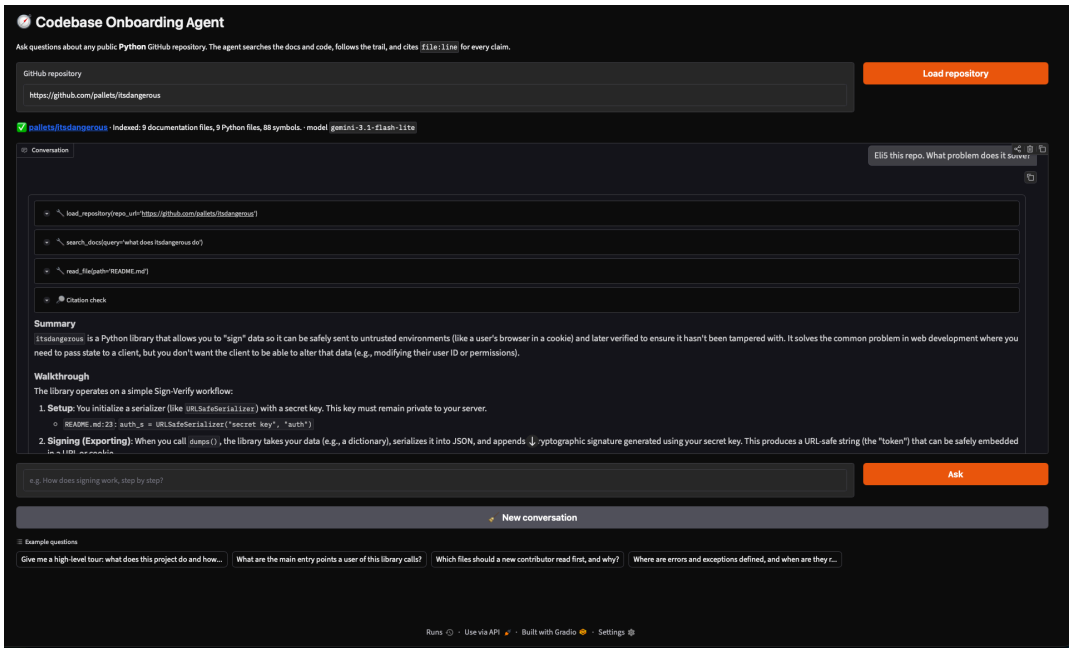


Figure 3: The Gradio web UI answering "ELI5 this repo". Each tool call and the citation check appear as collapsible steps above the structured answer.

Notebook run. Asked for a high-level tour, the agent called `load_repository`, `search_docs` and `list_files`, then drafted an answer citing bare filenames such as `signer.py`. The citation check flagged all six and suggested the full paths; the agent read `serializer.py` and `signer.py` and rewrote the answer, which passed with 11 verified citations in 9 seconds. The follow-up question (Figure 4) went the same way: a citation of a non-existent `requirements.txt` was sent back, the agent read `pyproject.toml` instead, and the answer passed with 7 verified citations in 8 seconds. The tools themselves return in well under a second.

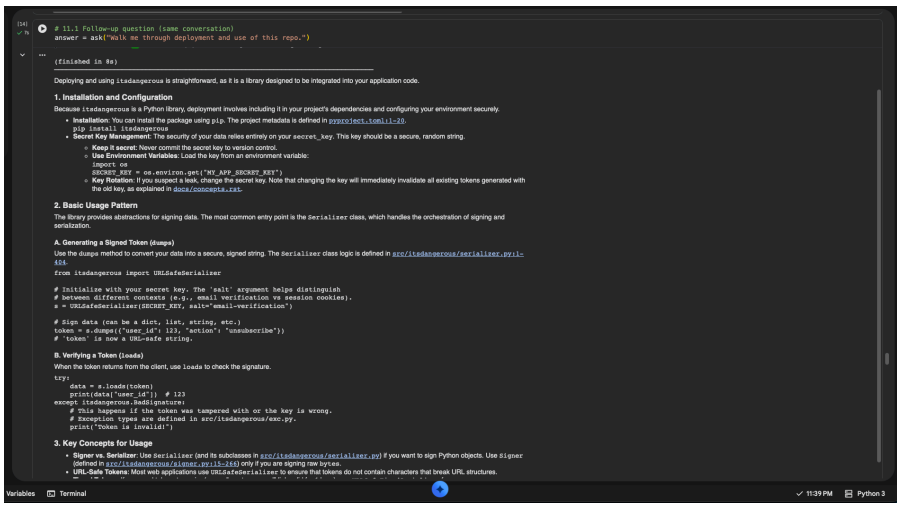


Figure 4: The follow-up question in the notebook, "Walk me through deployment and use of this repo." Citations link to the exact lines on GitHub.

Evaluation

Section 19 of the notebook asks eight questions about `pallets/itsdangerous`, each in a fresh conversation. A question passes only if the expected file is cited *and* every citation is valid; the out-of-scope question passes if the agent says the library doesn't do it. Results are saved to `eval_results.csv`. The run used `gemini-3.1-flash-lite`.

Question	Passes when the answer cites	Tool calls	Result
How does Signer create a signature, and how does it check one when unsigning?	`src/itsdangerous/signer.py`	3	✅
How do timestamps and `max_age` expiry work?	`src/itsdangerous/timed.py`	4	✅
Which exception is raised when a signature has expired, and where is it defined?	`src/itsdangerous/exc.py`	3	✅
How does `Serializer.dumps` turn a Python object into a signed string?	`src/itsdangerous/serializer.py`	6	✅
How does URLSafeSerializer make its output safe to put in a URL?	`url_safe.py` or `encoding.py`	4	✅
Which key derivation options does Signer support, and which one is the default?	`src/itsdangerous/signer.py`	3	✅
Where is the URL-safe base64 encoding and decoding implemented?	`src/itsdangerous/encoding.py`	2	✅
How does itsdangerous save signed tokens to a PostgreSQL database? *(out of scope)*	No file needed: the agent must say the library doesn't do this	2	✅

> **Result: 8/8 passed**, with all 37 citations valid and an average of 3.4 tool calls and 5 seconds per question. Answers often cited more than the expected file, such as `timed.py` and `__init__.py` alongside `exc.py`. The citation check and grading were also tested offline with a scripted stand-in model, covering a wrong path that was sent back and fixed, a line range still wrong after one rewrite, and the tool limit.

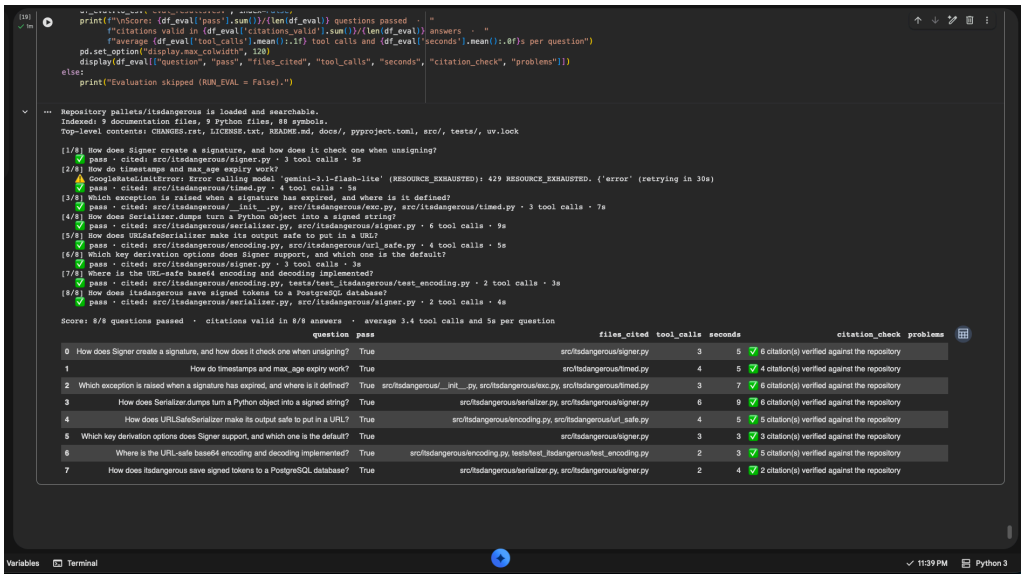


Figure 5: Evaluation output in the notebook.

Design challenges

Challenge	How it's handled
Gemini rate limits and overload (429 / 503 / 504)	The free tier allows only 30,000 embedding tokens per minute. Requests are grouped by estimated tokens, and a rolling-minute rate limiter waits *before* each call; on a 429 the uploader waits as long as Google asks and slows down. At start-up each chat model is probed, and the ones that respond form a fallback chain.
Interrupted indexing	Chunk IDs are deterministic, so the loader compares the stored vector count with the expected count and embeds only the missing chunks.
Chunking code meaningfully	Fixed-size splitting would cut functions in half. AST-based chunks (one per function, method, class and module, each with a signature and docstring header) give search far better units.
Agent over-exploring	A tool-calling agent tends to read whole files and wander. The prompt asks for targeted line ranges, and the graph enforces a 15-call limit, after which the model must answer with tools switched off.
Wrong or invented citations	Models often cite bare filenames (`signer.py` instead of `src/itsdangerous/signer.py`), files that don't exist, or lines past the end of a file, which become broken links. The `check_citations` node verifies every citation and sends the answer back once, suggesting the real path.

Pinecone constraints	Index names are sanitised, the index dimension is detected from the embedding model, chunks stay under the metadata limit, and a short wait after each upload allows for eventual consistency.
Multi-user web UI	All tools share one "active repository". Each visitor gets their own conversation thread, and a lock stops two questions from switching repositories under each other.
Keeping secrets out of the repo	API keys and the web UI login are read from Colab Secrets, never written into the notebook or README.

Limitations & future work

- **Python only:** tree-sitter based chunking could add other languages.
- **No automatic refresh:** storing the commit hash per namespace would allow incremental re-indexing.
- **Semantic search only:** hybrid search (vector + keyword/BM25) could improve exact-identifier queries.
- **Small evaluation:** eight questions on one repository, checking citations rather than every sentence. More questions across more repositories would be more reliable.
- **Large repos** are capped at a configurable number of embedded chunks; the navigation tools still cover the whole repo.

How to use it

Setup: open the notebook in Colab, add `GOOGLE_API_KEY` and `PINECONE_API_KEY` (optionally `UI_USERNAME` / `UI_PASSWORD`) under **Secrets**, then **Runtime → Run all**.

Web interface: paste a GitHub link in the Gradio app, click **Load repository**, and chat.

In the notebook:

```
ask("your question", repo_url="https://github.com/owner/repo")
df = run_eval()   # the 8-question evaluation (section 19)
```